

קורס אבטחת מערכות ווב

Parameter Tampering Attack – POC

[\[github-link\]](#)

Evgeni Glushko

317562031

Azrieli College of Engineering

Jerusalem

תוכן ענייניים

תיאור החולשה והרקע התאורטי

- מהי החולשה
- כיצד היא נוצרת
- באיזה הקשר היא מופיעה במערכות ווב
- מה תוקף יכול להשיג באמצעותה

סביבת העבודה

- טכנולוגיות (שפת תכנות, סביבת עבודה מסד נתונים וכו...)
- כלים המשמשים לביצוע ההדגמה (אם יש כאלה)

מימוש והדגמת המתקפה (POC)

- תיאור של ה-POC
- שלבי הניצול (השלבים של המתקפה)
- Payloads / קוד / בקשות לדוגמה.
- תוצאה בפועל של הניצול
- דיאגרמה המתארת את שלבי המתקפה.

מנגנוני הגנה ומניעה

- מדוע ההגנות הקיימות נכשלו
- כיצד ניתן למנוע את החולשה
- Best Practices רלוונטיים

מקורות

- מאמרים
- תיעוד רשמי
- OWASP
- קישורים רלוונטיים

תיאור החולשה והרקע התאורטי

מהי החולשה

Parameter Tampering היא חולשת אבטחה המתרחשת כאשר השרת סומך על ערכים שמתקבלים מהלקוח (Client) מבלי לאמת אותם בצד השרת.

במילים פשוטות:

השרת מאמין לנתונים שהמשתמש שולח — גם אם המשתמש שינה אותם בזדון.

כיצד היא נוצרת

החולשה נוצרת כאשר:

- השרת מקבל פרמטרים מה-Client (form fields / query params / JSON).
- הערכים ניתנים לשינוי ע"י המשתמש (דרך Burp, DevTools, וכו').
- אין בדיקה/חישוב מחודש בצד השרת.
- מתבצע שימוש ישיר בערך שהגיע מהלקוח.

ב-POC שלנו:

```
client_total_raw = request.form.get("client_total", "0")  
charged = max(0, client_total) # <-- VULNERABLE
```

השרת משתמש ב-`client_total` במקום לחשב בעצמו את הסכום האמיתי.

באיזה הקשר היא מופיעה במערכות ווב

החולשה נפוצה במיוחד ב:

- חנויות אונליין (מחיר מוצר)
- שינוי `role (user → admin)`
- שינוי כמות
- שינוי מזהה משתמש
- שינוי סכום תשלום
- שינוי `discount`

כל מקום שבו יש:

- Hidden input
- Query parameter
- JavaScript variable
- Cookie value

מה תוקף יכול להשיג באמצעותה

במערכת החנות: (P-Coins Ice Cream Shop)

התוקף יכול:

- לשלם 1 P-coin במקום 110

- לקבל מוצרים כמעט בחינם

- לעקוף מנגנון תשלום

בעולם אמיתי זה עלול לגרום ל:

- הפסדים כספיים

- Fraud

- Privilege Escalation

- שינוי נתונים קריטיים

סביבת העבודה

טכנולוגיות

- Backend: Python 3
- Framework: Flask 3.0.3
- Signing library: itsdangerous 2.2.0
- Session-based wallet
- "In-memory "database

קובץ ראשי: app.py

מימוש והדגמת המתקפה (POC)

תיאור ה-POC

נבנתה אפליקציית חנות גלילות המדמה מערכת תשלום ב-P-Coins.

המערכת כוללת:

- Cart
- Wallet
- Checkout
- תשלום Vulnerable
- תשלום Fixed

שלבי הניצול

שלב 1 – הוספת מוצרים לסל משתמש מוסיף פריטים לסל (לדוגמה: 4 גלילות).

שלב 2 – מעבר ל-Checkout

השרת מציג:

- server_total
- client_total (hidden field)

שלב 3 – שינוי פרמטר בצד הלקוח

פותחים DevTools → משנים:

```
<input name="client_total" value="110">
```

ל-: 1

שלב 4 – שליחת הבקשה

נשלחת בקשת POST ל: pay/vuln/

עם: client_total=1

שלב 5 – השרת סומך על הלקוח

$charged = \max(0, client_total)$

השרת מחייב רק 1 P-coin במקום 110.

Payload לדוגמה

בקשה מקורית

POST /pay/vuln

client_total=110

בקשה לאחר שינוי

POST /pay/vuln

client_total=1

תוצאה בפועל של הניצול

- היתרה יורדת ב-1 בלבד

- הסל מתנקה

- העסקה נחשבת מוצלחת

זה מוכיח שהשרת סומך על ערך שניתן לשינוי.

דיאגרמת שלבי המתקפה

User → Browser → Modify Parameter → Send Request



Server trusts value



Undercharged payment

מנגנוני הגנה ומניעה

מדוע ההגנות הקיימות נכשלו

- השרת לא מחשב סכום מחדש
- אין בדיקת שלמות נתונים
- אין חתימה קריפטוגרפית
- אין הפרדה בין UI לבין לוגיקה עסקית

כיצד ניתן למנוע את החולשה

1. Never trust the client

תמיד לחשב סכומים בצד השרת: $charged = server_total$

2. חישוב מחדש של כל ערך רגיש

מחיר מוצר חייב להילקח מה־DB בלבד.

3. חתימה קריפטוגרפית

ב-POC שתמשנו ב: URLSafeSerializer

השרת מוודא שה־`cart_token` לא שונה.

4. Input validation בלבד לא מספיק

בדיקת range לא מונעת tampering לוגי.

דרכי עבודה מומלצות - Best Practices

- Never trust client-side validation
- Recalculate everything server-side
- Use signed tokens
- Store sensitive data only server-side
- Implement audit logging
- Use CSRF protection
- Apply Principle of Least Privilege

השוואה בין Vulnerable ל-Fixed

Feature	Vulnerable	Fixed
Uses client_total	✓	✗
Recalculate server_total	✗	✓
Signed cart token	✗	✓
Tampering detectable	✗	✓

מקורות

OWASP

OWASP Top 10

Broken Access Control

[/https://owasp.org/Top10](https://owasp.org/Top10)

OWASP Testing Guide

Parameter Tampering (Testing for Parameter Manipulation)

[/https://owasp.org/www-project-web-security-testing-guide](https://owasp.org/www-project-web-security-testing-guide)

תיעוד רשמי

Flask Documentation

[/https://flask.palletsprojects.com](https://flask.palletsprojects.com)

itsdangerous Documentation

[/https://itsdangerous.palletsprojects.com](https://itsdangerous.palletsprojects.com)

קישורים רלוונטיים

- OWASP Web Security Testing Guide
- PortSwigger Academy – Parameter Tampering
- CWE-472: External Control of Assumed-Immutable Web Parameter